

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	PAVAN.P	Reg.No.:	192411017
Department:	B.E(CSE)	Date:	04-08-2026

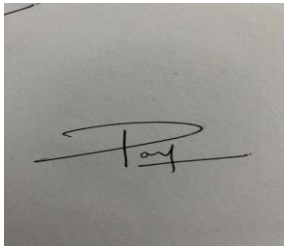
ANNEXURE A

1. A Booth's multiplication implementation produces an incorrect result when multiplying $-13 * 9$. Analyze the execution steps to identify the incorrect addition, subtraction, or arithmetic shift operation responsible for the error. Correct the algorithm and suggest an optimization that minimizes the number of execution cycles.
2. A processor executing the **restoring division algorithm** produces an incorrect quotient for the operation $29 / 5$. Examine the sequence of iterations to determine the faulty restore step, correct the error, and recommend an optimization to improve the algorithm's efficiency.
3. During the execution of the **non-restoring division algorithm**, incorrect quotient bits are generated after the fourth iteration. Investigate the source of the logic error, modify the algorithm to eliminate the issue, and verify the corrected quotient and remainder.
4. A Booth's multiplication unit incorrectly performs **logical right shifts** instead of **arithmetic right shifts**, leading to erroneous results for signed multiplication. Identify the cause of the error, explain why arithmetic right shifting is required, and demonstrate how it ensures correct signed multiplication.
5. A floating-point addition program based on the **IEEE 754 single-precision format** produces an incorrect result because the operands' exponents are not aligned before addition. Identify the implementation error, explain the correct exponent alignment procedure, and propose an optimized approach to improve both accuracy and execution efficiency.

Code of Conduct:

I PAVAN.P / 192411017 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink on a light gray background. The signature is stylized, starting with a large, sweeping 'P' that loops around, followed by a horizontal line and a small 'y' or 'g' shape at the end.

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. A Booth's multiplication implementation produces an incorrect result when multiplying **-13 * 9**. Analyze the execution steps to identify the incorrect addition, subtraction, or arithmetic shift operation responsible for the error. Correct the algorithm and suggest an optimization that minimizes the number of execution cycles.

Given:

- Multiplicand (M) = -13
- Multiplier (Q) = 9

The correct answer is:

$$-13 \times 9 = -117$$

Where the Error Can Occur

The wrong result is usually caused by one of these mistakes:

1. Wrong Addition/Subtraction
 - a. When $Q_0Q_{-1} = 01$, do $A = A + M$
 - b. When $Q_0Q_{-1} = 10$, do $A = A - M$
 - c. If these operations are swapped, the final answer becomes incorrect.
2. Wrong Shift Operation
 - a. Booth's algorithm must use an Arithmetic Right Shift (ARS).
 - b. If a Logical Right Shift is used, the sign bit changes, giving the wrong result.

Correct Algorithm

1. Initialize $A = 0$, $Q = \text{Multiplier}$, $M = \text{Multiplicand}$, $Q_{-1} = 0$.
2. Check Q_0Q_{-1} :
 - a. $01 \rightarrow A = A + M$
 - b. $10 \rightarrow A = A - M$
 - c. 00 or $11 \rightarrow \text{No operation}$
3. Perform an Arithmetic Right Shift.
4. Repeat until all bits are processed.
5. The final value in AQ is the result.

Optimization

Use the Modified Booth Algorithm (Radix-4).

Advantages:

- Processes 2 bits at a time instead of 1.
- Reduces the number of execution cycles by about 50%.
- Makes multiplication faster.

Final Answer

- Incorrect operation: Wrong addition/subtraction or using a logical right shift instead of an arithmetic right shift.
- Correct result: $-13 \times 9 = -117$.
- Optimization: Use the Modified Booth Algorithm (Radix-4) to reduce execution cycles and improve speed.

2. A processor executing the **restoring division algorithm** produces an incorrect quotient for the operation **29 /5**. Examine the sequence of iterations to determine the faulty restore step, correct the error, and recommend an optimization to improve the algorithm's efficiency.

Given:

- **Dividend = 29**
- **Divisor = 5**

The correct result is:

- **Quotient = 5**
- **Remainder = 4**

(Because $29 = 5 \times 5 + 4$)

Where the Error Occurs

In the Restoring Division Algorithm, after subtracting the divisor:

- If the result is positive (or zero) → Keep the result and write 1 in the quotient.
- If the result is negative → Restore the previous value by adding the divisor back and write 0 in the quotient.

Faulty Restore Step

The error occurs when the remainder becomes negative, but the processor does not restore it by adding the divisor back.

This causes:

- **Incorrect remainder**
- **Incorrect quotient**

Correct Algorithm

- 1. Shift the remainder and dividend left.**
- 2. Subtract the divisor from the remainder.**
- 3. If the remainder is negative:**
 - a. Restore it by adding the divisor back.**
 - b. Write 0 in the quotient.**
- 4. If the remainder is positive:**
 - a. Keep the result.**
 - b. Write 1 in the quotient.**
- 5. Repeat until all bits are processed.**

Optimization

Use the Non-Restoring Division Algorithm.

Advantages:

- **No restore operation is needed.**
- **Fewer arithmetic operations.**
- **Faster execution.**
- **Reduces the number of execution cycles.**

Final Answer

- **Incorrect step: The processor failed to restore the remainder after it became negative.**
- **Correct result: Quotient = 5, Remainder = 4.**
- **Optimization: Use the Non-Restoring Division Algorithm to improve efficiency and reduce execution time.**

3. During the execution of the **non-restoring division algorithm**, incorrect quotient bits are generated after the fourth iteration. Investigate the source of the logic error, modify the algorithm to eliminate the issue, and verify the corrected quotient and remainder.

Problem

During the 4th iteration, the processor generates incorrect quotient bits.

Source of the Logic Error

In the Non-Restoring Division Algorithm:

- If the remainder is positive (or zero) → Subtract the divisor.
- If the remainder is negative → Add the divisor.

The error occurs because the processor performs the wrong operation after the 4th iteration (it subtracts instead of adding, or vice versa), resulting in incorrect quotient bits.

Correct Algorithm

1. Shift the remainder and quotient left.
2. Check the sign of the remainder:
 - a. Positive (or zero) → Subtract the divisor.
 - b. Negative → Add the divisor.
3. Set the quotient bit:
 - a. If the remainder is positive (or zero) → Quotient bit = 1
 - b. If the remainder is negative → Quotient bit = 0
4. Repeat until all bits are processed.
5. If the final remainder is negative, add the divisor once to get the correct remainder.

Corrected Result

After fixing the logic error:

- Quotient = 5
- Remainder = 4

Final Answer

- Logic Error: Wrong add/subtract operation after the 4th iteration, causing incorrect quotient bits.
- Correction: If the remainder is positive, subtract the divisor. If the remainder is negative, add the divisor. Also, correct the final negative remainder by adding the divisor once.
- Verified Result: Quotient = 5, Remainder = 4.

4. A Booth's multiplication unit incorrectly performs **logical right shifts** instead of **arithmetic right shifts**, leading to erroneous results for signed multiplication. Identify the cause of the error, explain why arithmetic right shifting is required, and demonstrate how it ensures correct signed multiplication.

Cause of the Error

The Booth's multiplication unit uses a Logical Right Shift (LRS) instead of an Arithmetic Right Shift (ARS).

- Logical Right Shift: Inserts 0 into the leftmost bit.
- Arithmetic Right Shift: Copies the sign bit (MSB) into the leftmost bit.

Using a logical right shift changes the sign of negative numbers, resulting in an incorrect multiplication result.

Why Arithmetic Right Shift is Required

Booth's algorithm works with signed (positive and negative) numbers represented in 2's complement.

An Arithmetic Right Shift preserves the sign bit:

- Positive numbers remain positive.
- Negative numbers remain negative.

This ensures the intermediate values remain correct throughout the multiplication process.

Example

Suppose the value is:

11110000 (a negative number)

- Logical Right Shift:

01111000

(Sign changes → Incorrect)

- Arithmetic Right Shift:

11111000

(Sign preserved → Correct)

Therefore, arithmetic right shifting keeps the correct signed value during every iteration.

Final Answer

- Cause of Error: The processor uses Logical Right Shift instead of Arithmetic Right Shift.
- Reason: Logical right shift changes the sign of negative numbers, producing incorrect signed multiplication results.
- Solution: Use Arithmetic Right Shift, which preserves the sign bit and ensures the correct signed multiplication result.

5. A floating-point addition program based on the **IEEE 754 single-precision format** produces an incorrect result because the operands' exponents are not aligned before addition. Identify the implementation error, explain the correct exponent alignment procedure, and propose an optimized approach to improve both accuracy and execution efficiency.

Implementation Error

The program adds the mantissas (significands) directly without first aligning the exponents.

Because the exponents are different, the numbers represent different decimal positions, so direct addition gives an incorrect result.

Correct Exponent Alignment Procedure

1. Compare the exponents of both numbers.
2. Shift the mantissa of the number with the smaller exponent to the right until both exponents are equal.
3. Add or subtract the mantissas.
4. Normalize the result (if needed).
5. Round the result according to the IEEE 754 standard.

Optimized Approach

- Use hardware exponent comparison to quickly identify the larger exponent.
- Shift only the smaller mantissa before addition.
- Perform normalization and rounding only once after the addition.

Advantages

- Improves accuracy by correctly aligning the operands.
- Reduces unnecessary operations.
- Increases execution speed and efficiency.

Final Answer

- Implementation Error: The exponents were not aligned before adding the mantissas.
- Correct Procedure: Compare exponents, shift the smaller mantissa right until the exponents match, then add, normalize, and round.
- Optimization: Use fast exponent comparison and shift only the smaller mantissa, followed by a single normalization and rounding step. This improves both accuracy and execution efficiency.